

Merge Sort

Haimon Cugler Vieira¹, João Alexandre Pinto Camargo², Jocieli Pontes Domingues da Silva³, Kevin da Silva Oliveira⁴

¹Faculdade de Tecnologia do Estado de São Paulo

²Faculdade de Tecnologia do Estado de São Paulo

³Faculdade de Tecnologia do Estado de São Paulo

⁴Faculdade de Tecnologia do Estado de São Paulo

*haimon.vieira@aluno.cps.sp.gov.br, joao.camargo33@fatec.sp.gov.br,
jocieli.domingues@aluno.cps.sp.gov.br, kevin.oliveira6@aluno.cps.sp.gov.br*

1 Introdução

Os algoritmos de ordenação são fundamentais na área da computação, sendo utilizados para organizar dados de forma eficiente. Entre os diversos métodos existentes, o Merge Sort se destaca por sua eficiência e desempenho consistente [1].

O Merge Sort é um algoritmo baseado na técnica de **divisão e conquista**, onde o problema é dividido em partes menores, resolvido individualmente e posteriormente combinado.

Este trabalho tem como objetivo apresentar o funcionamento do algoritmo Merge Sort, sua implementação em linguagem C e sua importância na computação.

2 Funcionamento do Merge Sort

O Merge Sort funciona em três etapas principais: dividir, conquistar e combinar. Como mostrado na Figura 1, o algoritmo divide e combina os elementos.

2.1 Dividir

O vetor é dividido em duas partes aproximadamente iguais até que cada subvetor contenha apenas um elemento.

2.2 Conquistar

Cada subvetor, contendo apenas um elemento, já está naturalmente ordenado.

2.3 Combinar

Os subvetores são intercalados (merge), comparando seus elementos e organizando-os em ordem crescente.

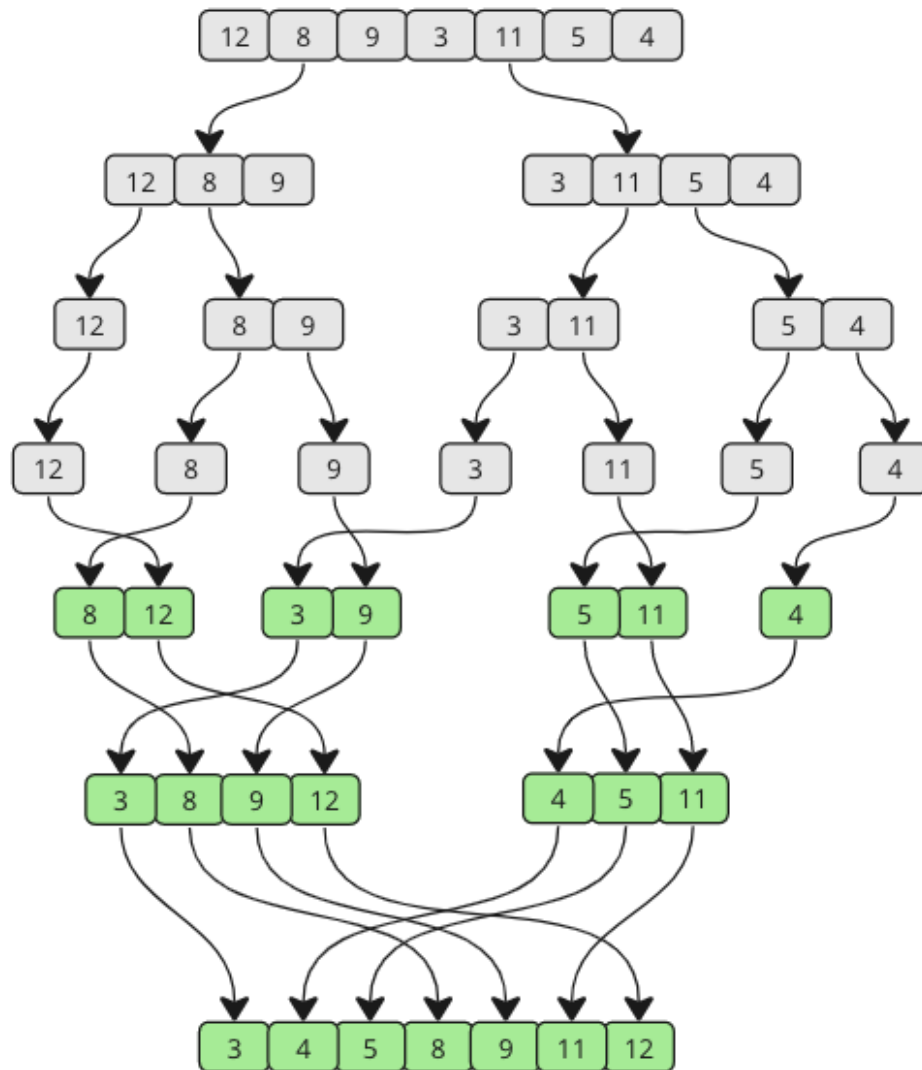


Figura 1: Processo de divisão e combinação do Merge Sort
Fonte: W3Schools.

3 Complexidade do Algoritmo

O Merge Sort apresenta as seguintes complexidades:

- Melhor caso: $O(n \log n)$
- Caso médio: $O(n \log n)$
- Pior caso: $O(n \log n)$

Isso significa que o algoritmo mantém um desempenho eficiente independentemente da entrada.

4 Notação Big-O

A notação Big-O é utilizada para descrever a complexidade de algoritmos, indicando como o tempo de execução cresce em relação ao tamanho da entrada [2].

No caso do Merge Sort, a complexidade é $O(n \log n)$, onde n representa o número de elementos e $\log n$ corresponde ao número de divisões realizadas no vetor.

Isso ocorre porque o algoritmo divide o problema recursivamente em partes menores e, em seguida, realiza a combinação dos elementos, resultando em um crescimento proporcional a $n \log n$.

Dessa forma, o Merge Sort apresenta um desempenho eficiente mesmo para grandes volumes de dados [3].

5 Implementação em C

O código apresentado a seguir foi retirado de um exemplo disponível no site GeeksforGeeks [4], que demonstra a implementação do algoritmo Merge Sort em linguagem C.

```
/*
Código retirado de:
GeeksforGeeks - Merge Sort
https://www.geeksforgeeks.org/dsa/merge-sort/
*/
function merge(arr, left, mid, right) {
    const n1 = mid - left + 1;
    const n2 = right - mid;

    // Create temp arrays
    const L = new Array(n1);
    const R = new Array(n2);

    // Copy data to temp arrays L[] and R[]
    for (let i = 0; i < n1; i++)
        L[i] = arr[left + i];
    for (let j = 0; j < n2; j++)
        R[j] = arr[mid + 1 + j];

    let i = 0, j = 0;
    let k = left;

    // Merge the temp arrays back into arr[left..right]
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {
            arr[k] = L[i];
            i++;
        } else {
            arr[k] = R[j];
            j++;
        }
        k++;
    }

    // Copy the remaining elements of L[], if there are any
    while (i < n1) {
        arr[k] = L[i];
```

```
        i++;
        k++;
    }

    // Copy the remaining elements of R[], if there are any
    while (j < n2) {
        arr[k] = R[j];
        j++;
        k++;
    }
}

function mergeSort(arr, left, right) {
    if (left >= right)
        return;

    const mid = Math.floor(left + (right - left) / 2);
    mergeSort(arr, left, mid);
    mergeSort(arr, mid + 1, right);
    merge(arr, left, mid, right);
}

// Driver code
const arr = [38, 27, 43, 10];
mergeSort(arr, 0, arr.length - 1);
console.log(arr.join(" "));
```

6 Vantagens e Desvantagens

6.1 Vantagens

- Alta eficiência: $O(n \log n)$
- Estabilidade na ordenação
- Ideal para grandes volumes de dados

6.2 Desvantagens

- Uso de memória adicional
- Implementação mais complexa

7 Conclusão

O Merge Sort é um dos algoritmos de ordenação mais eficientes, sendo amplamente utilizado na computação. Sua abordagem baseada em divisão e conquista garante desempenho consistente, tornando-o ideal para aplicações que exigem alto desempenho [5].

Apesar de utilizar memória adicional, suas vantagens tornam sua aplicação extremamente relevante em diversos contextos computacionais.

Referências

- [1] Informatica Nova Cruz, “Lógica do algoritmo de ordenação merge sort,” <https://www.youtube.com/watch?v=BnsYGiYYdnQ>, 2018, acesso em: 20 mar. 2026.
- [2] G. R. Mendes, “Algoritmos — merge sort,” 2022, acesso em: 23 mar. 2026. [Online]. Available: <https://guilherme-rmendes95.medium.com/algoritmos-merge-sort-ef12dadeba2a>
- [3] J. A. Bezerra, “Ordenação por comparação: Merge sort,” 2019, acesso em: 23 mar. 2026. [Online]. Available: <https://joaoarthurbm.github.io/eda/posts/merge-sort/>
- [4] GeeksforGeeks, “Merge sort,” 2025, Última atualização: 3 out. 2025. Acesso em: 23 mar. 2026. [Online]. Available: <https://www.geeksforgeeks.org/dsa/merge-sort/>
- [5] Clara Martinez, “Merge sort dance final 2 0,” https://www.youtube.com/watch?v=4NBxZozv3_A, 2017, acesso em: 16 mar. 2026.